

CSCI 580 Assignment 4

Solving the Traveling Salesman Problem with Simulated Annealing and a Genetic Algorithm

Geraldo Castro

Fall 2026

Code: github.com/Gcastro02/CSCI580_Geraldo_Castro (Assignment_4 folder)

1. Introduction

The Traveling Salesman Problem (TSP) asks for the shortest closed route that visits every city exactly once and then returns to the start. It sounds simple, but the number of possible tours grows really fast. With n cities there are $(n-1)!/2$ different tours, so for the 40 cities in this assignment that is about 1.0×10^{46} tours. Checking all of them is not an option, which is why heuristic methods like Simulated Annealing (SA) and Genetic Algorithms (GA) get used. They don't guarantee the best possible tour, but they can find a good one in a reasonable amount of time.

For this assignment I implemented both methods on the same problem instance: 40 cities placed randomly in the unit square, generated with `make_cities(n=40, seed=7)`. The goal was to beat the nearest-neighbor tour by at least 10%. After that, I added extra logging to both algorithms to compare how they behave while they run, and not only their final answers. All the numbers in this report come from my run of the notebook in Google Colab.

2. Problem Setup and Baseline

A tour is stored as a permutation of the city indices 0 to 39. Its length is the sum of the straight-line distances between consecutive cities, including the edge from the last city back to the first one.

The baseline is the nearest-neighbor (NN) heuristic. It starts at city 0 and always moves to the closest city it hasn't visited yet. This gave a tour length of **6.0214**. The left plot in Figure 1 shows that the NN tour has a few long edges that cut across the rest of the tour. That happens because the greedy choice leaves some far-away cities for the end. To pass the 10% requirement, a method has to get below $6.0214 \times 0.9 = 5.4193$.

3. Methods

3.1 Simulated Annealing

SA starts from a single tour (the NN tour here) and keeps trying small random changes to it. The change it uses is a 2-opt move. A 2-opt move picks two positions i and k and reverses the part of the tour between them. This removes two edges and reconnects the tour with two new ones, which makes it a good move for getting rid of edges that cross each other.

For every candidate tour I compute $\Delta = \text{cand_len} - \text{cur_len}$. If the candidate is better or the same length ($\Delta \leq 0$), it is always accepted. If it is worse, it is still accepted with probability $\exp(-\Delta / T)$. This is the main idea behind SA. When the temperature T is high, SA accepts a lot of worse moves and can wander around the search space. As T goes down, it gets pickier, until it basically only accepts improvements. These are the two parts I filled in:

```

accept = False
if delta <= 0:
    accept = True    # better or equal: always take it
elif rng.random() < math.exp(-delta / T):
    accept = True    # worse: take it with probability exp(-delta/T)

# ... later in the loop, geometric cooling:
T = max(T * cfg.alpha, 1e-12)

```

The `max(..., 1e-12)` part keeps `T` from ever reaching exactly 0, which would cause a divide-by-zero inside `exp(-delta / T)`. With `t0 = 0.2` and `alpha = 0.9995`, the temperature ends at about 9.1×10^{-6} after 20,000 iterations.

3.2 Genetic Algorithm

The GA works with a whole population of 200 tours instead of a single tour. The NN tour is included as one of them, and the other 199 are random. Every generation, the 5 best tours (the elites) are copied straight into the next generation. The rest of the new population is made by choosing two parents, combining them with crossover, and sometimes mutating the child. The main loop was already given, and I wrote the three helper functions below.

Tournament selection picks $k = 5$ random tours from the population and returns the shortest of them. Better tours get a higher chance of becoming parents, but a weaker tour can still win if none of the strong ones happen to be in its tournament.

```

def tournament_select(pop, lengths, k, rng):
    contenders = rng.sample(range(len(pop)), k)
    winner = min(contenders, key=lambda i: lengths[i])
    return pop[winner][:]

```

Order crossover (OX) is a crossover designed for permutations. A normal one-point crossover would create tours with repeated or missing cities. OX first copies a random slice of parent 1 into the child, in the same positions. It then fills the empty spots with parent 2's cities in the order they show up in parent 2, starting right after the slice and wrapping around, and skips any city that is already in the child. This way the child is always a valid tour and keeps part of the ordering from each parent.

```

def order_crossover_ox(parent1, parent2, rng):
    n = len(parent1)
    a, b = sorted(rng.sample(range(n), 2))
    child = [None] * n
    child[a:b + 1] = parent1[a:b + 1]
    used = set(child[a:b + 1])
    pos = (b + 1) % n
    for i in range(n):
        city = parent2[(b + 1 + i) % n]
        if city not in used:
            child[pos] = city
            used.add(city)
            pos = (pos + 1) % n
    return child

```

Swap mutation picks two random positions and swaps the cities in them. It is applied to 20% of the children.

```
def mutate_swap(tour, rng):
    t = tour[:]
    i, j = rng.sample(range(len(t)), 2)
    t[i], t[j] = t[j], t[i]
    return t
```

All three functions return a copy instead of changing a tour in place. This matters because the parents are still part of the population, and changing them by accident would mess up the population for the rest of the generation.

4. Experimental Setup

Table 1. Parameters used for the main runs (the defaults from the notebook).

Algorithm	Parameter	Value
SA	Iterations	20,000
SA	Initial temperature t_0	0.2
SA	Cooling rate α	0.9995
SA	Seed / starting tour	123 / NN tour
GA	Population size	200
GA	Generations	400
GA	Tournament size k	5
GA	Crossover rate / mutation rate	0.9 / 0.2
GA	Elite size	5
GA	Seed / starting population	999 / NN tour + 199 random tours

For Part 6 I made copies of the two functions, `sa_instrumented` and `ga_instrumented`. They run exactly the same algorithms but log extra data along the way:

- **SA:** the delta of every accepted move and every rejected move, plus the current length, best length and temperature at every iteration.
- **GA:** the best length, the population average length, and the number of unique tours in the population at every generation. I used the number of unique tours as a simple way to measure diversity.

To compare SA and GA fairly, I counted tour evaluations instead of iterations. One SA iteration evaluates one tour, but one GA generation evaluates 200 tours (the whole population). The full SA run used 20,001 evaluations and the full GA run used 80,200. I also ran both algorithms with 10 different seeds (1 to 10) to see how much the result depends on the random seed.

5. Results

Table 2. Results of the main runs.

Method	Best length	Improvement vs NN	Evals to within 1% of final	Total evals
Nearest neighbor	6.0214	baseline	n/a	n/a
Simulated Annealing	5.0057	16.87%	6,580	20,001
Genetic Algorithm	5.3672	10.86%	3,000	80,200

Both methods passed the 10% requirement, but the GA only passed by less than one percentage point.

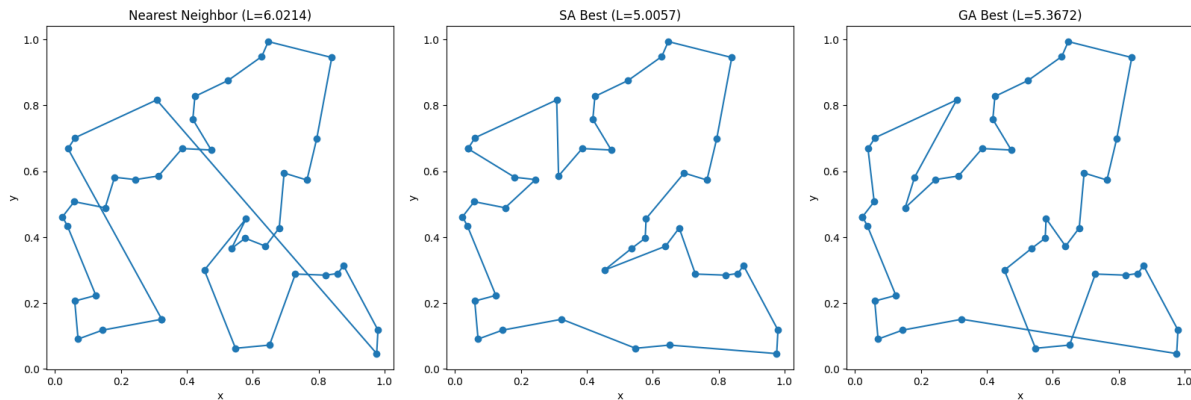


Figure 1. Nearest-neighbor tour (left), best SA tour (middle) and best GA tour (right).

Figure 1 shows the three tours. I couldn't find any crossing edges in the SA tour. The GA tour still has one crossing near the bottom, where the long edge going over to the bottom-right corner crosses another edge. A single 2-opt move would remove that crossing and make the tour shorter, but swap mutation can't really do that. I think this is a good example of why the GA got stuck (more on that in Section 6).

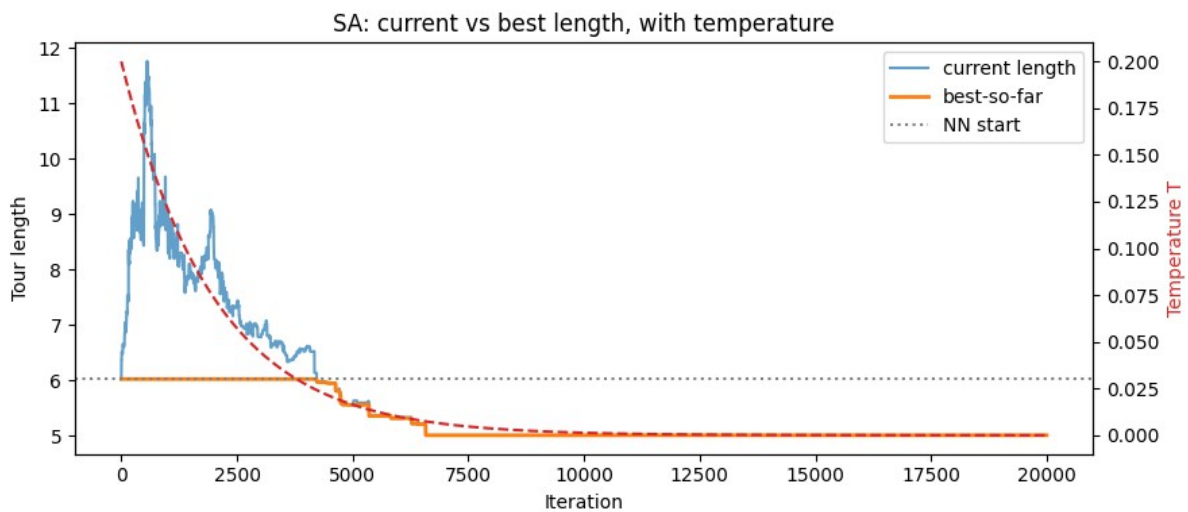


Figure 2. SA current tour length and best-so-far length, with the temperature on the right axis.

Figure 2 shows how SA behaves over time. At the start the temperature is high, so SA accepts a lot of worse moves, and the current tour actually gets much worse than the NN tour it started from. It peaks at almost 12 around iteration 600. Because of this, the best-so-far stays at 6.0214 for roughly the first 4,300 iterations. Once the temperature drops to around 0.02, the current tour starts improving and falls below the starting length, and the best-so-far drops quickly to 5.0057 by about iteration 6,600. After that, T is so small that SA basically turns into hill climbing and the tour doesn't change anymore.

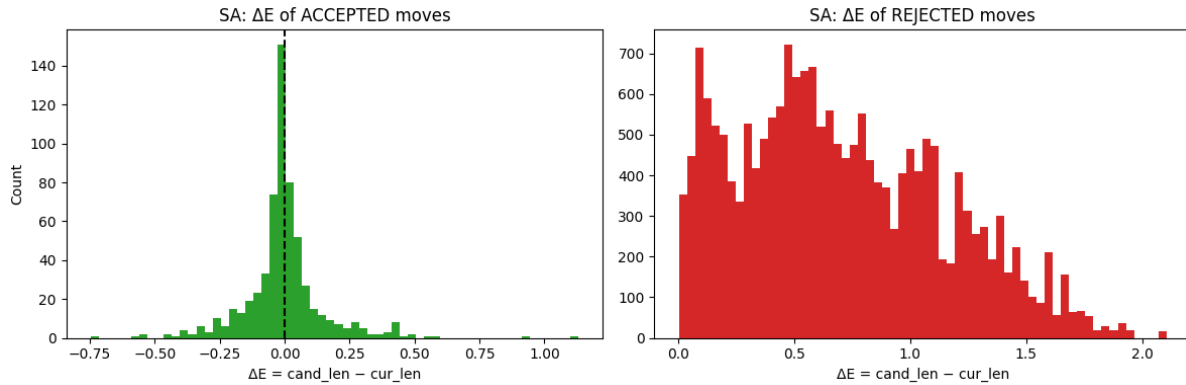


Figure 3. Histograms of delta for accepted SA moves (left) and rejected SA moves (right).

Out of 20,000 SA moves, 608 were accepted and 19,392 were rejected, so only about 3% of moves were accepted. Of the accepted moves, 283 were worse than the current tour (delta > 0). In Figure 3, the accepted histogram is centered right around 0. Improving moves are on the left, and worse moves that SA took anyway are on the right, a few of them with delta above 0.5 and one around 1.1. The rejected histogram only has positive values, which makes sense because improving moves are always accepted. It goes up to about 2.1, and most rejected moves have a delta between 0 and 1.5. These are random reversals that would mess up a decent tour, and the histogram shows that most random 2-opt moves are bad ones.

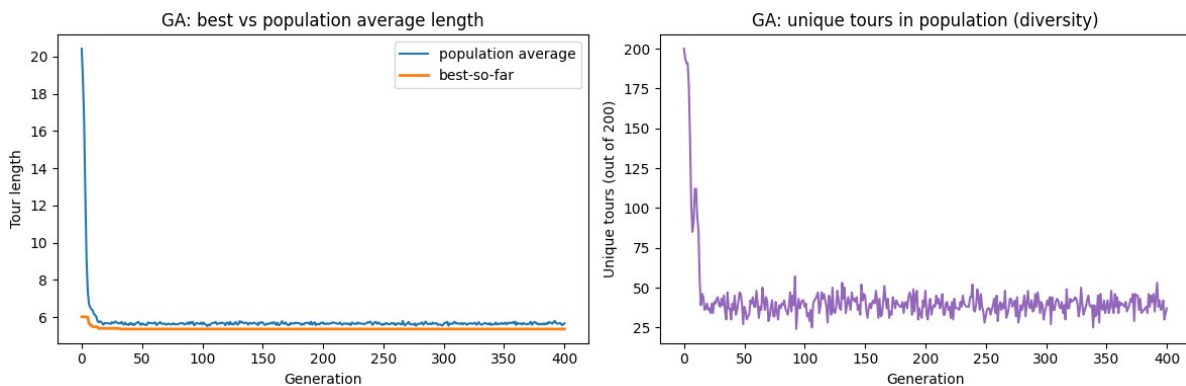


Figure 4. GA best-so-far and population average length (left) and number of unique tours in the population (right).

Figure 4 shows the GA. The population average starts at about 20, because random tours are really bad, and drops to around 5.7 within about 10 to 15 generations. After that it stays flat, just above the best line. The best-so-far reaches 5.3672 a little after generation 30 and never improves again. The number of unique tours drops from 200 to below 50 within the first 15 or so generations. It was 38 at generation 50 and 37 at generation 400, and it bounces between about 25 and 55 for the rest of the run.

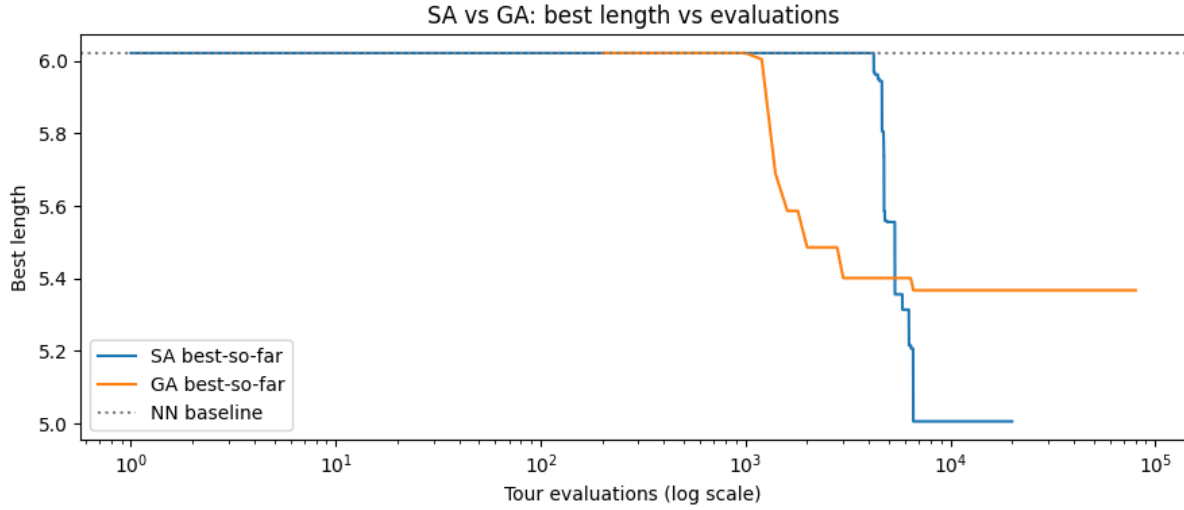


Figure 5. Best-so-far length for SA and GA vs number of tour evaluations (log scale).

Figure 5 puts both algorithms on the same x-axis using the number of evaluations. The GA starts improving first, after about 1,000 evaluations (around generation 5), and gets within 1% of its final value after about 3,000 evaluations. SA doesn't improve its best tour until around 4,300 evaluations, but once it starts it drops quickly and ends lower, at 5.0057, after about 6,600 evaluations. In both cases, everything after that point (about 13,000 SA evaluations and about 74,000 GA evaluations) didn't improve the best tour at all.

Table 3. Best tour length over 10 seeds (1 to 10) with the default parameters.

Method	Mean	Std dev	Min	Max
SA	4.9700	0.1359	4.8339	5.2197
GA	5.5537	0.3523	5.1546	5.9001

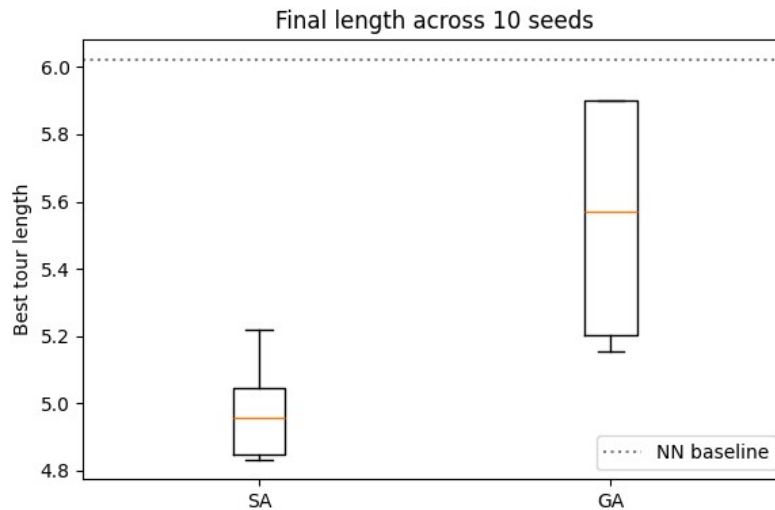


Figure 6. Final tour length across 10 seeds. The dotted line is the NN baseline.

Table 3 and Figure 6 show the 10-seed runs. Every SA run passed the 10% cutoff, and even the worst SA run (5.2197) was 13.3% better than NN. The GA was a lot less reliable. Its median was around 5.57, which is above the 5.4193 cutoff, so at least half of the GA runs would have failed `grade_check`. The worst GA run (5.9001) was only about 2% better than NN. Even the best GA run (5.1546) was worse than the average SA run.

6. Analysis

6.1 Which algorithm converged faster?

It depends on what "converged" means. The GA settled down faster. It was within 1% of its final value after about 3,000 evaluations, compared to about 6,580 for SA. The problem is that it settled on a worse tour (5.3672 vs 5.0057) and then spent the rest of its 80,200 evaluations without improving. SA took longer to settle because its high starting temperature makes it explore instead of improve. Even so, it found a better tour and used only a quarter as many total evaluations. So my answer is that the GA converged faster, but too early (premature convergence). SA converged more slowly but to a better answer.

6.2 Which showed more variability?

I looked at two kinds of variability. Within a single run, SA is much more variable. Its current tour jumps around between about 6 and almost 12 during the first few thousand iterations, because it accepts worse moves on purpose. The GA's best and average curves are pretty smooth by comparison after the first 10 or so generations.

Across different seeds it's the other way around. The GA's standard deviation was 0.3523, compared to 0.1359 for SA, which is about 2.6 times bigger. GA results ranged from 5.15 to 5.90, so whether the GA passes or fails depends a lot on the seed. SA ranged from 4.83 to 5.22, and every run passed. So SA is noisy while it runs but consistent in its final answer, and the GA is the opposite.

6.3 Did SA escape local minima? Evidence?

Yes. A pure hill climber starting from the NN tour can only accept improvements. It would stop as soon as no single 2-opt move makes the tour shorter. SA got past that. The evidence:

- 283 of the 608 accepted moves were worse than the current tour. They are the right side of the accepted histogram in Figure 3.
- In Figure 2 the current tour went up to almost 12, about twice the starting length. At iteration 4,000 it was still worse than where it started (6.5972 vs 6.0214), yet the best tour ended at 5.0057. To get from the NN tour to the final tour, SA had to pass through a lot of worse tours first, which a hill climber can't do.
- The NN tour had long crossing edges, and the final SA tour has none (Figure 1).

One thing I noticed is that the logging says the last worse move was accepted at iteration 19,009. At that point T was only about 1.5×10^{-5} , so only moves that are worse by a really tiny amount have any real chance of being accepted. In Figure 2, the current and best lines sit right on top of each other from about iteration 7,000 on. So the real "escaping" happens in roughly the first 7,000 iterations, and after that SA acts like hill climbing.

6.4 Did GA lose diversity? Evidence?

Yes, and it happened quickly. The evidence:

- The number of unique tours dropped from 200 to below 50 within about 15 generations, and stayed around 37 to 38 for the rest of the run (Figure 4). That means around 80% of the population was duplicates.
- The population average dropped from about 20 to about 5.7 and then stayed flat, just above the best line. When the average is that close to the best, most of the population has to be copies or small variations of the same few tours.
- The best tour stopped improving a little after generation 30, so the last 370 or so generations didn't help at all.

I think this happens for a few reasons. A tournament size of 5 plus 5 elites puts a lot of selection pressure on the best tours, so they take over the population quickly. Once most of the population is the same, OX between two nearly identical parents just gives back basically the same tour, so crossover stops producing anything new. That leaves swap mutation as the only source of new tours. Swapping two random cities in a good tour replaces up to four edges, and that almost always makes the tour worse, so the mutated children lose their tournaments and the population stays stuck. The leftover crossing in the GA tour is a good example of this. A 2-opt reversal would fix it in one move, but random swaps are very unlikely to.

6.5 Which worked better and why?

SA worked better on this problem. Its best tour was shorter (5.0057 vs 5.3672), its average over 10 seeds was much better (4.97 vs 5.55), it was more consistent across seeds, and it used fewer evaluations. I think the main reason is the move it uses. The 2-opt move fits the TSP really well because it removes two edges and reconnects the tour, which is exactly what you need to uncross edges. The GA's swap mutation and OX crossover don't work on edges in that way, and the GA lost most of its diversity before it found a really good tour.

If I worked on the GA more, I would try replacing swap mutation with an inversion (2-opt style) mutation, raising the mutation rate, or using smaller tournaments so the population keeps more diversity. Any of these might help the GA get closer to SA.

7. Conclusion

Both Simulated Annealing and the Genetic Algorithm beat the nearest-neighbor baseline by more than 10% with the default settings. SA reached 5.0057 (16.87% better) and the GA reached 5.3672 (10.86% better). The extra logging showed why they behave so differently. SA uses its high starting temperature to accept worse tours and get out of local minima, then cools into hill climbing and ends with a tour with no crossing edges. The GA improves quickly at first, but its population loses most of its diversity within about 30 generations, and swap mutation isn't strong enough to get it moving again. Running 10 seeds showed that SA is also a lot more reliable. For this TSP instance, SA was clearly the better choice.